Benemérita Universidad Autónoma de Puebla - date

Facultad de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

ALGORITMO DE AGRUPAMIENTO K-MEANS

Docente Pedro Tecanhuehue Vera

Hecho por:
Adylene Baylon Cuahtlapantzi
Karla Irais De Florencio Romero
Maria Fernanda Flores Ramos

17 de marzo de 2026

Índice

1. Introducción **2**

2. Descripción del algoritmo K-Means **2**
2.1. ¿Qué es K- means? 2
2.2. ¿Cómo funciona el algoritmo K- means? 2

3. Metodología **3**

4. Pruebas **4**
4.1. Resultados del Algoritmo Propio 4
4.1.1. Resultados generales de las ejecuciones 4
4.1.2. Análisis de las matrices de distancia 6
4.1.3. Análisis de las asignaciones de clusters 7
4.1.4. Interpretación de la gráfica de clusters 8
4.2. Resultados del Algoritmo usando Orange 9
4.2.1. Análisis del agrupamiento 9
4.2.2. Interpretación de errores 11
4.3. Conclusiones 11
4.4. APÉNDICES 11
A. CÓDIGO DEL ALGORITMO 11
B. EXPLICACIÓN DE TÉRMINOS 18
C. MANUAL DE USUARIO 19
D. REFERENCIAS 23
E. MANUAL TÉCNICO 23

1. Introducción

En la vida diaria solemos agrupar cosas sin darnos cuenta, como separar ropa por tipo o clasificar contactos según su relación con nosotros. De forma similar, el algoritmo K-Means permite agrupar datos en diferentes conjuntos, organizándolos según qué tan parecidos son entre sí. Por eso, es importante conocer este algoritmo y saber cómo aplicarlo correctamente, como veremos en este reporte.

2. Descripción del algoritmo K-Means

2.1. ¿Qué es K- means?

El algoritmo k-means es un método de agrupamiento que divide un conjunto de datos en k grupos o clusters. Los datos se agrupan de tal manera que los puntos en el mismo clúster sean más similares entre sí que los puntos en otros clusters.
Del universo de algoritmos de aprendizaje no supervisado, K-means sigue siendo uno de los algoritmos más conocidos para el aprendizaje no supervisado.

2.2. ¿Cómo funciona el algoritmo K- means?

Para utilizar el algoritmo K-means, primero se especifica el número de clusters deseados (k). Por ejemplo, al establecer ≪k≫ igual a 2, su conjunto de datos se agrupará en 2 grupos, mientras que si establece ≪k≫ igual a 4, agrupará los datos en 4 grupos.
Cada grupo está representado por su centro o centroide, que corresponde a la media aritmética de los puntos de datos asignados al grupo. De esta manera, el algoritmo funciona a través de un proceso iterativo hasta que cada punto de datos está más cerca del centroide de su propio grupo que de los centroides de otros grupos, minimizando la distancia dentro del grupo en cada paso.
Cómo funciona paso a paso el algoritmo K- means:

1. **Especificar el número de clústers deseados (k):** El primer paso es especificar cuántos clúster queremos dividir el conjunto de datos. Este número se denomina k.

2. **Seleccionar k puntos al azar del conjunto de datos como los centroides iniciales de cada clúster:** Luego, se eligen k puntos al azar del conjunto de datos para servir como los centroides iniciales de cada clúster. Estos centroides son el punto central o el promedio de cada clúster.

3. **Asignar cada punto del conjunto de datos al cluster cuyo centroide esté más cerca:** A continuación, el algoritmo asigna cada punto del conjunto de datos al cluster cuyo centroide esté más cerca. Para hacer esto, se calcula la distancia entre cada punto y cada centroide y se asigna el punto al cluster cuyo centroide tenga la menor distancia.

$$d(x, c) = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

   Esta fórmula representa la distancia entre un punto y un centroide, la cual se utiliza para determinar a qué cluster pertenece cada dato.

4. **Recalcular los centroides de cada cluster como la media de todos los puntos del cluster:** Una vez que todos los puntos han sido asignados a un cluster, se recalculan los centroides de cada cluster como la media de todos los puntos del cluster. Esto significa que se actualiza la posición del centroide para reflejar la nueva agrupación.

$$c_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

   Esta fórmula indica que el centroide se calcula como el promedio de todos los puntos que pertenecen a un cluster.

5. **Repetir los pasos 3 y 4** hasta que los centroides de los clusters ya no cambien o hasta que se alcance el número máximo de iteraciones.

3. Metodología

Para la implementación de este algoritmo, el problema se atacó a través de los siguientes módulos:

1. **Carga y Exploración de Datos:** Se importó la base de datos Breast Cancer del repositorio UCI. Asimismo, se describieron sus atributos (categóricos en su mayoría) sin aplicar ninguna transformación previa.

   a) Descripción de atributos
   La base contiene atributos clínicos como:
   - Edad
   - Menopausia
   - Tamaño del tumor
   - Número de nodos afectados
   - Irradiación
   - Clase (no usada en clustering)

2. **Aislamiento de Clases:** Se separó temporalmente la etiqueta de clasificación (clase) del resto de los datos, ya que K-Means es un algoritmo no supervisado y no debe estar influenciado por la etiqueta durante el agrupamiento.

3. **Inicialización de Centroides:** Se configuró el algoritmo para buscar $k = 2$ grupos, correspondientes a "no recurrence eventsz recurrence events". Se seleccionaron aleatoriamente 2 instancias existentes en la base de datos como centroides iniciales.

4. **Agrupamiento con HEOM:** Se programó el cálculo de distancias utilizando la métrica HEOM (Heterogeneous Euclidean-Overlap Metric) para manejar adecuadamente la naturaleza de los atributos.

5. **Evaluación:** Se contrastaron los 2 clústeres generados contra las etiquetas originales apartadas en el paso 2 para generar una Matriz de Confusión. A partir de esta, se calcularon las métricas de Sensibilidad, Exactitud, Precisión y Tasa de Error.

6. **Simulación:** Todo el proceso de agrupamiento y evaluación se introdujo en un ciclo de 20 iteraciones. Los resultados de cada iteración, junto con sus promedios, se exportaron al archivo EVALUACIÓN.XLSX.

4. Pruebas

4.1. Resultados del Algoritmo Propio

Para evaluar el desempeño del algoritmo K-Means implementado, se realizaron veinte ejecuciones independientes utilizando diferentes centroides iniciales seleccionados de manera aleatoria. Este procedimiento permite analizar el comportamiento del modelo ante distintas condiciones iniciales, considerando que K-Means es un algoritmo no supervisado.

4.1.1. Resultados generales de las ejecuciones

Los resultados obtenidos en las 20 ejecuciones se almacenaron en el archivo Excel generado automáticamente por el sistema.

Ejecucion	Iteraciones	VP	FP	VN	FN	Sensibilidad	Exactitud	Precision	Tasa_Error
1	3	39	88	113	46	0.458823529	0.531468531	0.307086614	0.468531469
2	4	37	33	168	48	0.435294118	0.716783217	0.528571429	0.283216783
3	4	54	125	76	31	0.635294118	0.454545455	0.301675978	0.545454545
4	4	48	110	91	37	0.564705882	0.486013986	0.303797468	0.513986014
5	4	51	116	85	34	0.6	0.475524476	0.305389222	0.524475524
6	5	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
7	2	43	101	100	42	0.505882353	0.5	0.298611111	0.5
8	3	43	86	115	42	0.505882353	0.552447552	0.333333333	0.447552448
9	3	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
10	4	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
11	4	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
12	3	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
13	4	51	116	85	34	0.6	0.475524476	0.305389222	0.524475524
14	2	52	106	95	33	0.611764706	0.513986014	0.329113924	0.486013986
15	2	48	110	91	37	0.564705882	0.486013986	0.303797468	0.513986014
16	3	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
17	4	47	111	90	38	0.552941176	0.479020979	0.297468354	0.520979021
18	2	54	125	76	31	0.635294118	0.454545455	0.301675978	0.545454545
19	3	43	86	115	42	0.505882353	0.552447552	0.333333333	0.447552448
20	2	37	33	168	48	0.435294118	0.716783217	0.528571429	0.283216783
PROMEDIO	3.25	46.45	100.6	100.4	38.55	0.546470588	0.513461538	0.328131249	0.486538462

Figura 1: Resultados obtenidos en las 20 iteraciones, almacenados en EVALUCION.XLSX

Se puede observar que:
Al analizar los resultados obtenidos, se puede observar que el algoritmo converge rápidamente, ya que en promedio tarda alrededor de 3 iteraciones en estabilizarse. Esto significa que encuentra una solución en poco tiempo, lo cual es una ventaja.
En cuanto al desempeño, la exactitud promedio fue de aproximadamente 51 %, lo que indica que el modelo acierta poco más de la mitad de los casos. Este resultado muestra que el algoritmo sí logra encontrar ciertos patrones en los datos, pero no es suficiente para separar completamente las clases.
La sensibilidad fue cercana al 54 %, lo que quiere decir que el modelo logra detectar un poco más de la mitad de los casos de recurrencia. Esto es importante porque esos casos son los más relevantes en el contexto médico. Sin embargo, la precisión fue de aproximadamente 32 %, lo que indica que muchos de los casos que el modelo predice como positivos en realidad no lo son. En otras palabras, hay varios falsos positivos.
Además, la tasa de error fue cercana al 48 %, lo que confirma que el modelo todavía comete una cantidad considerable de errores. Esto se debe principalmente a que los datos no están completamente separados, es decir, hay pacientes con características similares que pertenecen a diferentes clases.
Otro punto importante es que los resultados cambian entre ejecuciones. Por ejemplo, algunas ejecuciones alcanzan una exactitud más alta (como la ejecución 2 o la 20), mientras que otras tienen resultados más bajos. Esto demuestra que el algoritmo depende mucho de los centroides iniciales, y por eso es necesario ejecutar varias veces para obtener una evaluación más confiable.

4.1.2. Análisis de las matrices de distancia

Ejecucion	Matriz_Muestra	Tamanos
1	[[2.29, 1.5], [2. , 1.41], [2.24, 0.], [1.41, 2.24], [2. , 1.73]]	[159, 127]
2	[[2.06, 1.5], [2.65, 1.41], [2.24, 0.], [2.83, 2.24], [2.65, 1.73]]	[70, 216]
3	[[1.5 , 2.29], [1.41, 2.], [0. , 2.24], [2.24, 1.41], [1.73, 2.]]	[179, 107]
4	[[2.06, 1.8], [1. , 2.24], [1.41, 1.73], [1.73, 2.], [1.41, 2.24]]	[158, 128]
5	[[1.8 , 2.06], [2.24, 1.], [1.73, 1.41], [2. , 1.73], [2.24, 1.41]]	[167, 119]
6	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
7	[[1.41, 2.29], [1.8 , 1.73], [1.12, 2.], [2.29, 1.41], [1.8 , 2.]]	[144, 142]
8	[[2.24, 1.73], [2.06, 1.8], [2.29, 1.12], [1.12, 2.29], [2.06, 2.06]]	[157, 129]
9	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
10	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
11	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
12	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
13	[[1.8 , 2.06], [2.24, 1.], [1.73, 1.41], [2. , 1.73], [2.24, 1.41]]	[167, 119]
14	[[1.5 , 1.8], [1.41, 2.24], [0. , 1.73], [2.24, 2.], [1.73, 2.24]]	[158, 128]
15	[[2.06, 1.8], [1. , 2.24], [1.41, 1.73], [1.73, 2.], [1.41, 2.24]]	[158, 128]
16	[[2. , 1.8], [1.5 , 2.24], [1.8 , 1.73], [1.8 , 2.], [1.5 , 2.24]]	[128, 158]
17	[[1.8 , 2.], [2.24, 1.5], [1.73, 1.8], [2. , 1.8], [2.24, 1.5]]	[158, 128]
18	[[1.5 , 2.29], [1.41, 2.], [0. , 2.24], [2.24, 1.41], [1.73, 2.]]	[179, 107]
19	[[2.24, 1.73], [2.06, 1.8], [2.29, 1.12], [1.12, 2.29], [2.06, 2.06]]	[157, 129]
20	[[2.06, 1.5], [2.65, 1.41], [2.24, 0.], [2.83, 2.24], [2.65, 1.73]]	[70, 216]

Figura 2: Resultados de las matrices obtenidas en las 20 iteraciones, almacenados en EVALUCION.XLSX

En esta figura 2, se muestran las distancias entre cada paciente y los centroides. Estas distancias son las que usa el algoritmo para decidir a qué grupo pertenece cada paciente.
Se puede ver que algunos valores son pequeños, lo que significa que el paciente es muy parecido al centroide, mientras que otros valores son más grandes, indicando que hay más diferencia.
También se observa que las distancias cambian entre ejecuciones, ya que los centroides no siempre son los mismos. Esto es normal y es parte del comportamiento del algoritmo.
Además, hay casos donde un paciente tiene distancias parecidas a ambos centroides. Esto significa que no es tan claro a qué grupo pertenece, lo cual explica por qué el modelo comete errores en algunos

casos.

4.1.3. Análisis de las asignaciones de clusters

Ejecucion	Indice_Paciente	Cluster_Asignado	Distancia_Centroide_0	Distancia_Centroide_1
1	0	1	2.291287847	1.5
1	1	1	2	1.414213562
1	2	1	2.236067977	0
1	3	0	1.414213562	2.236067977
1	4	1	2	1.732050808
1	5	1	2	1.732050808
1	6	1	1.732050808	1.414213562
1	7	1	2.061552813	1.5
1	8	1	2.236067977	1
1	9	1	1.732050808	1.414213562
1	10	1	2.291287847	1.5
1	11	0	1.414213562	1.732050808
1	12	1	2.291287847	2.061552813
1	13	0	1.5	2.061552813
1	14	1	2.061552813	1.5
1	15	1	2.291287847	1.802775638
1	16	1	2.236067977	1
1	17	1	2.061552813	1.5
1	18	1	2.061552813	1.802775638
1	19	0	1.5	2.291287847
1	20	0	1.732050808	1.732050808

Figura 3: Asignación de clusters en las 20 iteraciones, almacenados en EVALUCION.XLSX

En esta tabla de la figura 3, se puede ver a qué grupo fue asignado cada paciente, junto con las distancias a los centroides.
Por ejemplo, si un paciente tiene menor distancia al centroide 1 que al centroide 0, entonces se asigna al grupo 1. Esto confirma que el algoritmo está funcionando correctamente en cuanto a la asignación. Sin embargo, también se observa que algunos pacientes cambian de grupo entre ejecuciones. Esto sucede porque están en una zona donde no hay una separación clara entre los grupos. Por otro lado, hay pacientes que siempre se asignan al mismo grupo, lo que indica que forman parte de un grupo bien definido.

Esto nos dice que los datos tienen partes donde los grupos están claros, pero también partes donde se mezclan.

4.1.4. Interpretación de la gráfica de clusters

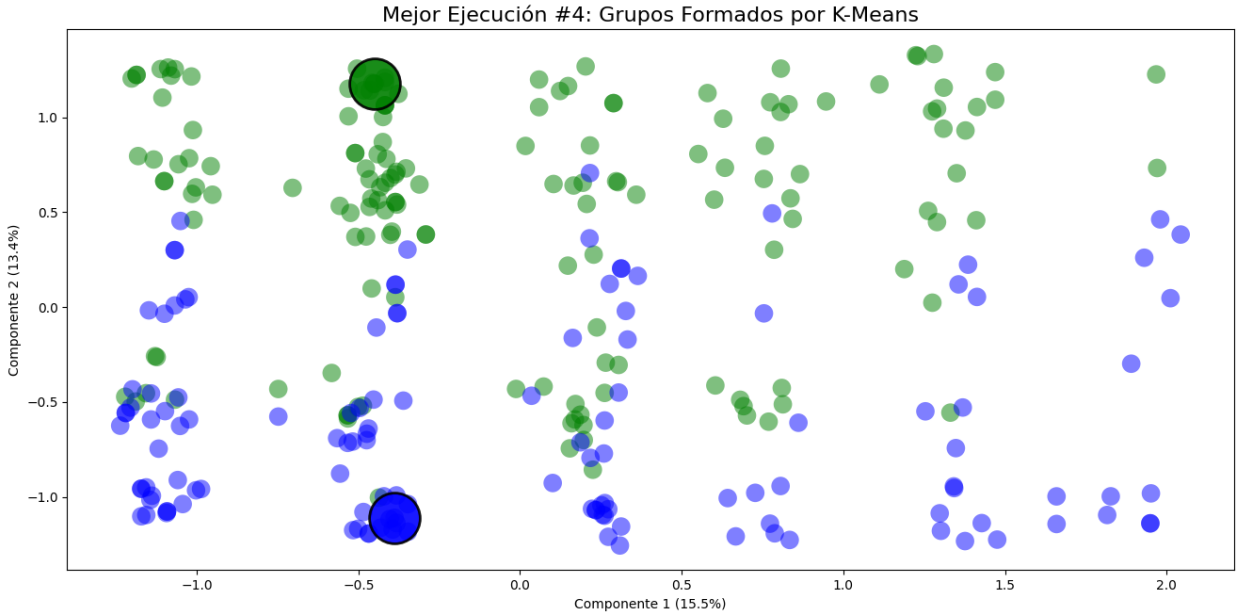

Figura 4: Ggráfica obtenida de clusters

La figura 4, muestra los grupos obtenidos por el algoritmo K-Means en su mejor ejecución. Cada punto representa un paciente y está coloreado según el grupo al que fue asignado. Los puntos verdes corresponden a un cluster y los azules a otro. Además, los círculos grandes indican la posición de los centroides de cada grupo.
Para poder visualizar los datos, se utilizó una técnica llamada PCA, la cual permite reducir la información a dos dimensiones. En este caso, la primera componente explica aproximadamente el 15.5 % de la variación de los datos y la segunda alrededor del 13.4 %.
Al observar la gráfica, se puede notar que existe una separación entre los grupos principalmente en la dirección vertical. Los puntos verdes se concentran más en la parte superior, mientras que los puntos azules se ubican en la parte inferior. Esto indica que el algoritmo logró identificar cierta diferencia entre los datos y formar dos grupos distintos.
Sin embargo, la separación no es completamente clara. En algunas zonas de la gráfica se pueden ver puntos de diferentes colores muy cercanos entre sí. Esto significa que hay pacientes con características similares que fueron asignados a diferentes grupos. Esta situación explica por qué el modelo no alcanza una exactitud alta.
También se puede notar que los grupos no tienen una forma completamente definida o uniforme, lo que sugiere que los datos no están organizados de manera simple. Esto representa una limitación para

el algoritmo K-Means, ya que funciona mejor cuando los grupos son más claros y separados.
En general, la gráfica confirma lo observado en los resultados numéricos: el modelo logra separar parcialmente los datos, pero existe cierta mezcla entre los grupos, lo cual afecta su desempeño.

4.2. Resultados del Algoritmo usando Orange

Para complementar el análisis, se utilizó el software Orange para aplicar el algoritmo K-Means sobre la misma base de datos. En este caso, fue necesario realizar una transformación de los datos, ya que el algoritmo en Orange trabaja únicamente con valores numéricos.
Las variables categóricas fueron convertidas a valores numéricos utilizando una codificación ordinal, donde a cada categoría se le asignó un número (por ejemplo: 1, 2, 3). En el caso de la clase, se representó como 0 y 1, lo cual permitió posteriormente comparar los resultados del agrupamiento. Figura 6

X.0	Cluster	Silhouette	X.1	X.2	X.3	X.4	X.5	X.6	X.7	X.8	X.9
no-recurrence-events recurrence-events	C1 C2	continuous	continuous	continuous	continuous	continuous	continuous	continuous	continuous	continuous	continuous
class	meta	meta									
no-recurrence-events	C2	0.56523	1	2	6	0	0	3	0	1	0
no-recurrence-events	C1	0.536714	2	2	4	0	0	2	1	4	0
no-recurrence-events	C1	0.57304	2	2	4	0	0	2	0	1	0
no-recurrence-events	C1	0.613832	4	0	3	0	0	2	1	2	0
no-recurrence-events	C1	0.620724	2	2	0	0	0	2	1	3	0
no-recurrence-events	C1	0.616059	4	0	3	0	0	2	0	1	0
no-recurrence-events	C2	0.505085	3	2	5	0	0	2	0	1	0
no-recurrence-events	C1	0.57541	4	0	4	0	0	1	0	1	0
no-recurrence-events	C2	0.60964	2	2	10	0	0	2	0	1	0
no-recurrence-events	C1	0.567879	2	2	4	0	0	2	1	2	0
no-recurrence-events	C1	0.618473	2	2	0	0	0	3	0	0	0
no-recurrence-events	C2	0.496469	3	0	5	0	0	2	0	1	0
no-recurrence-events	C1	0.601776	4	1	2	0	0	1	0	4	0
no-recurrence-events	C2	0.51989	3	0	5	0	0	3	0	4	0
no-recurrence-events	C2	0.581101	2	2	6	0	0	3	0	2	0
no-recurrence-events	C2	0.556129	4	1	6	0	0	1	0	1	0
no-recurrence-events	C1	0.618173	2	2	3	0	0	2	0	1	0
no-recurrence-events	C2	0.573105	3	2	6	0	0	3	0	1	0
no-recurrence-events	C2	0.562613	4	0	6	0	0	3	0	1	0

Figura 5: Excel modificado con resultado de kmeans

4.2.1. Análisis del agrupamiento

Después de aplicar K-Means con K=2, el modelo generó dos clusters. En la visualización de la figura 6 se representan con colores distintos:

- **Azul para Cluster 1**
- **Rojo para Cluster 2**

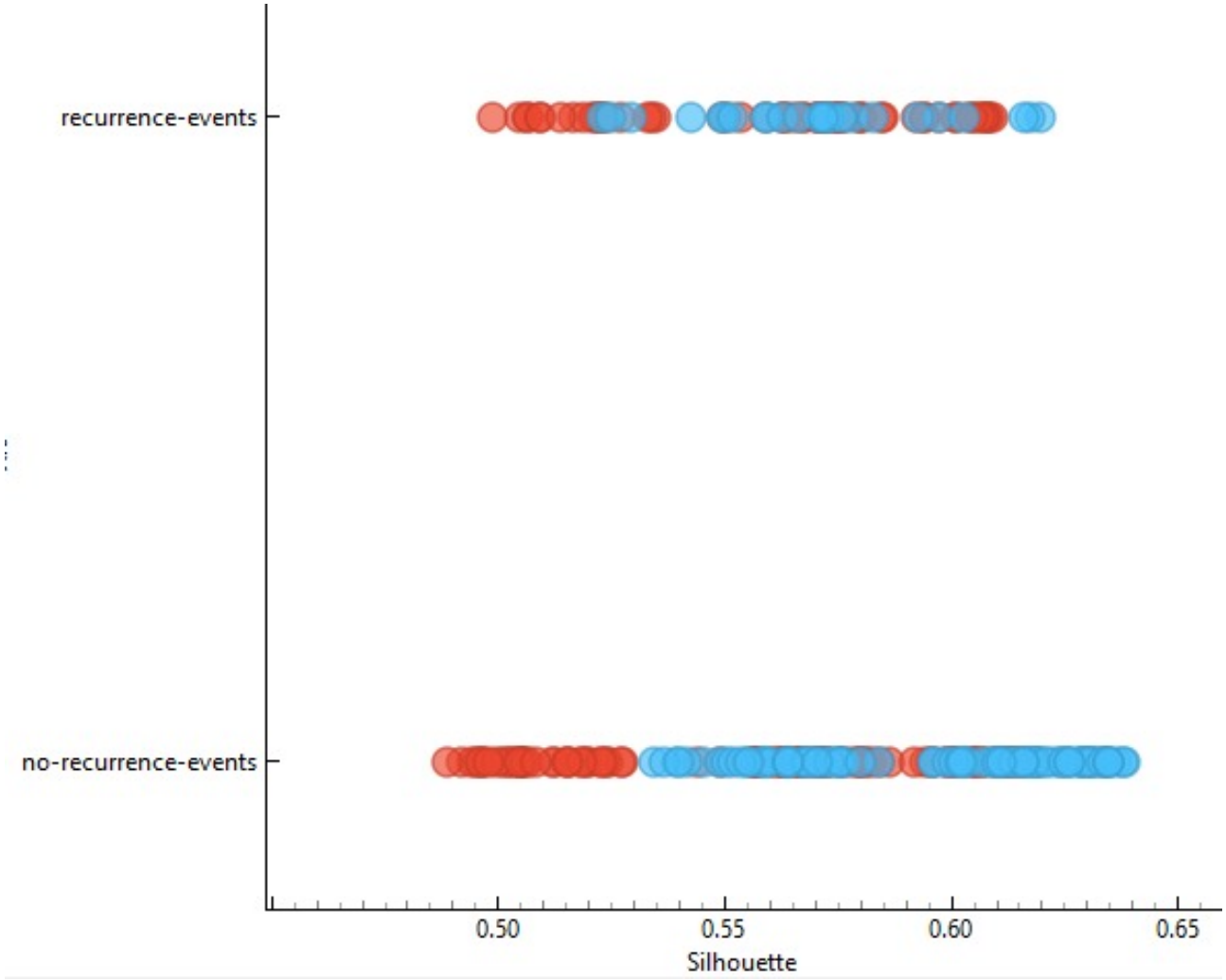

Figura 6: Gráfica obtenida de clusters usando Orange

Al observar la gráfica, se puede notar que los datos se agrupan en dos niveles horizontales, correspondientes a las clases reales:

- Parte superior es recurrence-events
- Parte inferior es no-recurrence-events

Esto permite comparar directamente cómo el clustering se relaciona con las clases reales.
También podemos ver que en la gráfica (figura 6) se muestra el valor de Silhouette para cada punto, el cual mide qué tan bien está clasificado dentro de su cluster.
Se observa que la mayoría de los valores se encuentran aproximadamente entre 0.50 y 0.65, lo cual indica que:

- Los puntos están razonablemente bien agrupados
- Existe cierta separación entre clusters

Sin embargo, también se observan algunos puntos con valores más bajos, lo que indica que esos casos están cerca del límite entre clusters y pueden estar mal agrupados.

4.2.2. Interpretación de errores

Al comparar los colores (clusters) con las clases reales, se puede notar que muchos puntos coinciden correctamente (cluster = clase), pero también hay varios casos donde el color no corresponde a la clase real.
Por ejemplo: los puntos rojos en la zona azul o viceversa, esto indica que el modelo también comete errores, similar a lo observado en la implementación propia.

4.3. Conclusiones

En esta práctica se implementó el algoritmo K-Means utilizando la métrica HEOM y se compararon los resultados con los obtenidos en Orange.
En la implementación propia en Python, se observó que el algoritmo logra agrupar los datos y encontrar cierta estructura sin usar la clase real. Sin embargo, la exactitud obtenida (alrededor del 51 %) muestra que la separación no es completamente clara. Esto se debe a que los datos presentan mezcla entre clases, lo que dificulta su agrupación. También se comprobó que el algoritmo depende de los centroides iniciales, ya que los resultados cambian entre ejecuciones.
Por otro lado, en Orange se obtuvieron resultados similares, donde también se logra una separación parcial de los datos. El uso del coeficiente Silhouette mostró que los clusters son aceptables, pero no perfectos. Además, la transformación de variables categóricas a números puede afectar los resultados, ya que introduce un orden que en realidad no existe.
En general, ambos enfoques permiten identificar patrones en los datos, pero también muestran que K-Means tiene limitaciones cuando los grupos no están bien definidos. Aun así, es una herramienta útil para explorar la información y entender mejor cómo están organizados los datos.

4.4. APÉNDICES

A. CÓDIGO DEL ALGORITMO

En este apartado se presentan los fragmentos más importantes del código desarrollado para la implementación del algoritmo K-Means utilizando la métrica HEOM, así como la evaluación del modelo.

- ***Cálculo de la distancia HEOM***

```python
# ----------------FÓRMULAS HEOM ----------------
def range_a(data):
    #Calcula el rango = max - min (usando nanmax/nanmin para ignorar huecos en el
    r = np.nanmax(data) - np.nanmin(data)
    return r if r != 0 else 1.0

rangos = {col: range_a(X[col]) for col in cols_num}

def overlap(x, y):
    #Distancia para atributos nominales. 0 si son iguales, 1 si son diferentes
    return 0.0 if x == y else 1.0

def rn_diff(x, y, r):
    #Distancia  para atributos numéricos |x-y|/rango
    return abs(x - y) / r if r != 0 else 0.0

def d_a(x, y, col, es_num, r=None):
    #Función general que decide qué fórmula usar según el tipo de atributo
    if es_num:
        return rn_diff(x, y, r)
    else:
        return overlap(x, y)
```

Figura 7: Parte 1 del código de HEOM en Python

```python
def calcular_heom(idx1, idx2, d_num, d_cat, rangos_dict, cols_c):
    #HEOM(x,y) = sqrt( suma (d_a)^2 )
    #Calcula la distancia total entre dos pacientes sumando las diferencias de todos sus atributos.
    #REGLA HEOM: Si un atributo es desconocido (NaN), su distancia es 1 (máxima).

    suma_cuadrados = 0.0
    #Parte numérica
    for col in rangos_dict:
        val1 = d_num[col][idx1]
        val2 = d_num[col][idx2]

        # Verificación explícita de valores faltantes para aplicar la regla de distancia=1
        if np.isnan(val1) or np.isnan(val2):
            d = 1.0
        else:
            d = d_a(val1, val2, col, True, rangos_dict[col])

        suma_cuadrados += d ** 2

    #Parte categórica
    for col in cols_c:
        val1 = d_cat[col][idx1]
        val2 = d_cat[col][idx2]

        #Verificación de valores faltantes (NaN o string 'nan')
        if pd.isna(val1) or pd.isna(val2) or str(val1) == 'nan' or str(val2) == 'nan':
```

Figura 8: Parte 2 del código de HEOM en Python

```python
            d = 1.0
        else:
            d = d_a(val1, val2, col, False)

        suma_cuadrados += d ** 2

    return np.sqrt(suma_cuadrados)
```

Figura 9: Parte 3 del código de HEOM en Python

En este fragmento se implementa la distancia HEOM, la cual permite calcular la similitud entre objetos con atributos numéricos y categóricos.

Para los atributos numéricos se utiliza una diferencia normalizada, mientras que para los categóricos se asigna 0 si los valores son iguales y 1 si son diferentes. Además, cuando existen

valores faltantes, se asigna una distancia máxima de 1.

Esta función es fundamental, ya que permite aplicar K-Means sin transformar los datos.

- ***Implementación del algoritmo K-Means***

```python
# ---------------- ALGORITMO K-MEANS ----------------
def kmeans_heom(X_df, k=2, max_iter=50):

    #Implementación del algoritmo K-Means usando HEOM:
    #1. Calcular distancia de cada paciente a los centroides actuales.
    #2. Generar los grupos asignando cada paciente al centroide más cercano.
    #3. Calcular nuevos centroides (primero teórico, luego buscando el objeto real más cercano).
    #4. Repetir hasta que los grupos no cambien (convergencia).

    n = len(X_df)
    #Prepara diccionarios con los valores en arrays para que el cálculo sea rápido
    d_num = {col: X_df[col].values for col in cols_num}
    d_cat = {col: X_df[col].values for col in cols_cat}

    #Inicialización aleatoria de los centroides
    centroides_idx = random.sample(range(n), k)

    historial = []
    errores = []

    for i in range(max_iter):
        etiquetas = np.zeros(n, dtype=int)
        matriz_dist = np.zeros((n, k))
        error_actual = 0.0
```

Figura 10: Parte 1 del código de K-Means en Python

```python
# ---------------- ALGORITMO K-MEANS ----------------
def kmeans_heom(X_df, k=2, max_iter=50):

    #Implementación del algoritmo K-Means usando HEOM:
    #1. Calcular distancia de cada paciente a los centroides actuales.
    #2. Generar los grupos asignando cada paciente al centroide más cercano.
    #3. Calcular nuevos centroides (primero teórico, luego buscando el objeto real más cercano).
    #4. Repetir hasta que los grupos no cambien (convergencia).

    n = len(X_df)
    #Prepara diccionarios con los valores en arrays para que el cálculo sea rápido
    d_num = {col: X_df[col].values for col in cols_num}
    d_cat = {col: X_df[col].values for col in cols_cat}

    #Inicialización aleatoria de los centroides
    centroides_idx = random.sample(range(n), k)

    historial = []
    errores = []

    for i in range(max_iter):
        etiquetas = np.zeros(n, dtype=int)
        matriz_dist = np.zeros((n, k))
        error_actual = 0.0
```

Figura 11: Parte 2 del código de K-Means en Python

En esta parte se realiza la asignación de los datos a los clusters. Primero se seleccionan centroides iniciales de forma aleatoria y posteriormente se calcula la distancia de cada objeto a cada centroide utilizando la función HEOM.

Cada objeto se asigna al cluster cuya distancia sea menor, formando así los grupos iniciales.

- ***Actualización de centroides***

```python
# CALCULAR NUEVOS CENTROIDES
nuevos_idx = []
for c in range(k):
    mask = (etiquetas == c)
    ids_cluster = np.where(mask)[0]

    if len(ids_cluster) == 0:
        nuevos_idx.append(random.randint(0, n-1))
        continue

    #1. Calcula el centroide "TEÓRICO" (Promedio de valores)
    # Para numéricos: Media aritmética (usamos nanmean para ignorar NaN al promediar).
    # Para categóricos: Moda (el valor que más se repite, ignorando NaN).
    teorico = {}
    for col in cols_num:
        teorico[col] = np.nanmean(d_num[col][ids_cluster])
    for col in cols_cat:
        # Filtramos valores válidos (no NaN) para calcular la moda correctamente
        validos = [v for v in d_cat[col][ids_cluster] if pd.notna(v) and str(v) != 'nan']
        if len(validos) > 0:
            u, cnt = np.unique(validos, return_counts=True)
            teorico[col] = u[np.argmax(cnt)]
        else:
            teorico[col] = 'desconocido'
```

Figura 12: Parte 3 del código de K-Means en Python

Aquí se recalculan los centroides. Para los atributos numéricos se calcula el promedio, mientras que para los categóricos se utiliza el valor más frecuente (moda).

Esto permite actualizar los centroides con base en los datos asignados a cada cluster.

- ***Evaluación del modelo***

```python
VP = np.sum((bin_p==1)&(bin_r==1))
FP = np.sum((bin_p==1)&(bin_r==0))
VN = np.sum((bin_p==0)&(bin_r==0))
FN = np.sum((bin_p==0)&(bin_r==1))

tot = VP+VN+FP+FN
if tot==0: return {'VP':0,'FP':0,'VN':0,'FN':0,'Sensibilidad':0,'Exactitud':0,'Precision':0,'Tasa_

return {
    'VP':int(VP), 'FP':int(FP), 'VN':int(VN), 'FN':int(FN),
    'Sensibilidad': VP/(VP+FN) if (VP+FN)>0 else 0,
    'Exactitud': (VP+VN)/tot,
    'Precision': VP/(VP+FP) if (VP+FP)>0 else 0,
    'Tasa_Error': (FP+FN)/tot
}
```

Figura 13: Parte 4 del código de K-Means en Python

En esta sección se calculan las métricas de evaluación comparando los clusters obtenidos con las clases reales.

Estas métricas permiten medir el desempeño del modelo, indicando qué tan bien logra aproximar la clasificación real.

- ***Ejecución 20 veces***

```python
#REPETIR EL PROCEDIMIENTO 20 VECES

for i in range(1, 21):
    print(f"\n--- Ejecución {i}/20 ---")
    try:
        #Llamamos a la función que hace todo el procesos (kmeans_heom)
        etq, cent, hist, errs = kmeans_heom(X, k=2)

        todos_errores.append(errs)
        todas_etiquetas.append(hist[-1]['etiquetas'])
        todos_centroides.append(hist[-1]['centroides_idx'])

        mets = evaluar(etq, y_real)
```

Figura 14: Parte 5 del código de K-Means en Python

En esta parte se ejecuta el algoritmo 20 veces para analizar su estabilidad. Esto es importante porque los resultados pueden cambiar dependiendo de los centroides iniciales.

B. EXPLICACIÓN DE TÉRMINOS

Aquí se describen los principales conceptos utilizados en la implementación del algoritmo.

- ***Algoritmo K-Means***

  K-Means es un algoritmo de agrupamiento que divide un conjunto de datos en K grupos, donde cada objeto pertenece al grupo cuyo centroide es más cercano.

  El algoritmo sigue estos pasos:

  1) Seleccionar K centroides iniciales de forma aleatoria.
  2) Calcular la distancia de cada objeto a los centroides.
  3) Asignar cada objeto al cluster más cercano.
  4) Recalcular los centroides con base en los objetos asignados.
  5) Repetir hasta que los centroides no cambien.

  El objetivo es minimizar la suma de distancias dentro de cada cluster, logrando que los elementos de un mismo grupo sean similares.

- ***Distancia HEOM (Heterogeneous Euclidean-Overlap Metric)***

  La distancia HEOM se utiliza para calcular la similitud entre objetos que tienen atributos de distintos tipos (numéricos y categóricos).

  Se define como la raíz cuadrada de la suma de las distancias de cada atributo:

  $$HEOM(x, y) = \sqrt{\sum d_a(x, y)^2}$$

  Donde cada atributo se calcula de la siguiente forma:

  - **Atributos numéricos:**

    $$d = \frac{|x - y|}{\text{rango}}$$

    Esto permite normalizar los valores para que todos tengan el mismo peso.

  - **Atributos categóricos:**

    $$d = \begin{cases} 0 & \text{si son iguales} \\ 1 & \text{si son diferentes} \end{cases}$$

  - **Valores faltantes:**

    $$d = 1$$

    Esto evita eliminar datos incompletos y permite trabajar directamente con ellos.

- ***Matriz de confusión***

  La matriz de confusión permite comparar los resultados del modelo con las clases reales.

  Se compone de:

- **VP (Verdaderos positivos):** casos positivos correctamente identificados
- **FP (Falsos positivos):** casos negativos clasificados como positivos
- **VN (Verdaderos negativos):** casos negativos correctamente identificados
- **FN (Falsos negativos):** casos positivos clasificados como negativos

- ***Métricas de evaluación***

  A partir de la matriz de confusión se calculan:

  - **Exactitud:**
  $$\frac{VP + VN}{\text{Total}}$$
  - **Precisión:**
  $$\frac{VP}{VP + FP}$$
  - **Sensibilidad:**
  $$\frac{VP}{VP + FN}$$
  - **Tasa de error:**
  $$\frac{FP + FN}{\text{Total}}$$

  Estas métricas permiten evaluar el desempeño del modelo y entender qué tan bien clasifica los datos.

C. MANUAL DE USUARIO

En esta practica se utilizo el programa .ºrange”para simular como el uso de kmeans con la base de datos dada. Orange es un software de mineria de datos muy sencillo elaborado con una interfaz simple como se muestra en la Figura 15

Figura 15: Hoja de trabajo dentro de orange

Al abrir Orange nos muestra una ventana inicial como se puede observar en la Figura 16 donde podemos abrir un archivo (reciente o no) o crear uno nuevo asi como opciones adicionales

Figura 16: Ventana inicial al abrir orange

En la parte lateral izquierda se muestran todos los objetos posibles que podemos utilizar para realizar el flujo junto a una barra de busqueda para buscar algun modulo en especifico.

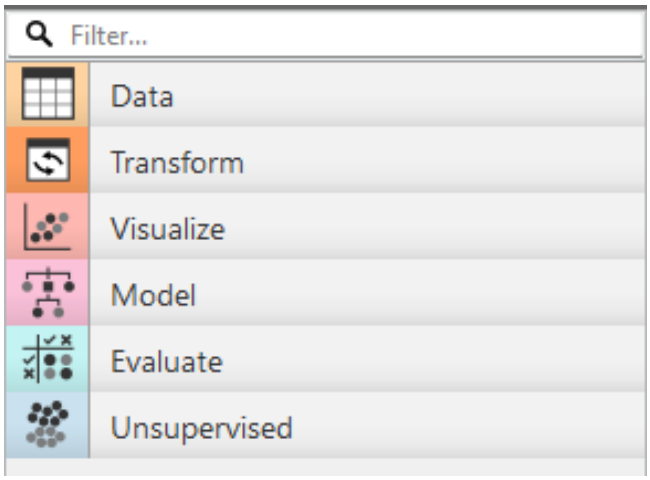

Figura 17: Secciones de los modulos dentro de orange

Como podemos ver en la Figura 17 esta tiene 6 secciones las cuales se explican a continuacion:

- **Data**: Contiene modulos para la carga y manejo inicial de datos, este incluye herramientas para

importar archivos (Excel, CSV, bases de datos), ver y explorar datos y seleccionar subconjuntos

- **Transform**: Contiene modulos para limpiar y preparar los datos antes del análisis.
- **Visualize**: Contiene modulos para representar los datos gráficamente.
- **Model**: Contiene modulos donde se construyen modelos de machine learning.
- **Evaluate**: Contiene modulos para medir qué tan bueno es un modelo.
- **Unsupervised**: Contiene modulos que se enfocan en modelos de aprendizaje no supervisado (sin etiquetas).

Para crear, guardar o abrir un archivo dentro de orange cuando hemos cerrado la ventana inicial nos dirigimos a la opcion "File"de la barra superior donde podemos encontrar las distintas opcioned de guardado y abrir un archivo como se muestra en la Figura 18

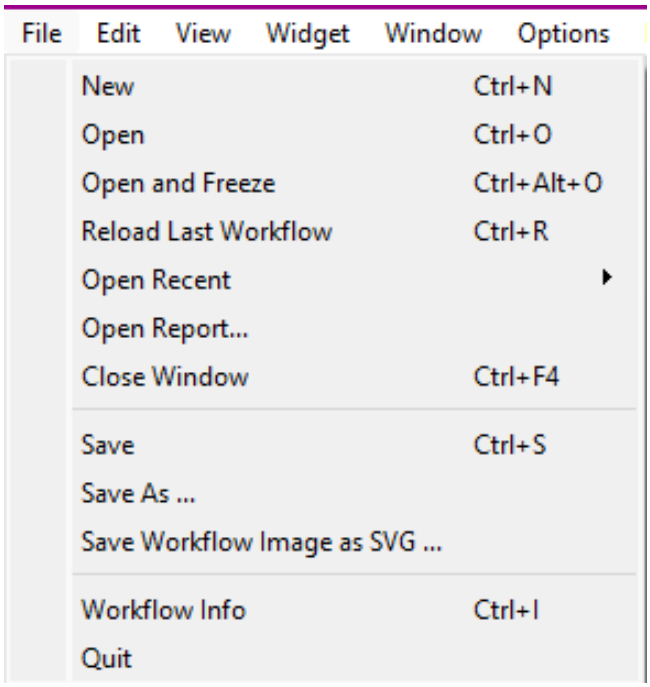

Figura 18: Opciones de abrir, guardar y crear un archivo en orange

Para colocar un nuevo modulo simplemente buscamos y arrastramos el modulo deseado a la hoja de trabajo, para explicar un poco mas sobre como se usan algunos modulos utilizare el modulo de Evaluate - Test and score. Este modulo tiene varias entradas (asi como muchos otros) y si damos doble click sobre el lado "("nos mostrara estas distintas entradas en las que nosostros seremos capaces de decidir exactamente a cual pertenece como se muestra en la figura 19

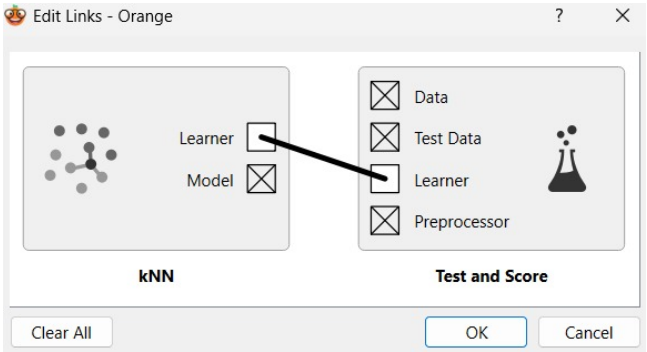

Figura 19: Demostracion de las distintas conexiones que puede tener un modulo

En este caso tenemos el modulo de entrada solo puede ser conectador con el "Learner"debido a que el modulo es un modelo. y sus salida es de tipo "Learner". Si fuera de tipo "Data"podria entrar en cualquiera de los dos primeros recuadros. Por ultimo agregar que asi como otros softwares en este tambien se pueden revisar las especificaciones y configuraciones de los modulos dando doble click sobre ellos. Basicamente esto es todo lo que requerimos saber para crear cualquier modelo dentro de Orange.

D. REFERENCIAS

En este apartado se presentan las fuentes utilizadas para el desarrollo de la práctica.

1) Base de datos

   Breast Cancer Dataset. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/14/breast+cancer

2) Artículo de la distancia HEOM

   Wilson, D. R., & Martinez, T. R. (1997). *Improved Heterogeneous Distance Functions*. Journal of Artificial Intelligence Research, 6, 1–34.

E. MANUAL TÉCNICO

En este apartado se describen los elementos técnicos necesarios para la ejecución del programa desarrollado, así como las herramientas utilizadas.

- ***Lenguaje de programación***

  El algoritmo fue desarrollado en Python, debido a su facilidad para el manejo de datos y disponibilidad de bibliotecas para análisis y procesamiento.

  Bibliotecas utilizadas:

  Para la implementación del algoritmo se utilizaron las siguientes bibliotecas:

  - **pandas:** utilizada para la lectura y manipulación de datos desde archivos CSV.

- **numpy (si lo usaste):** para operaciones numéricas y manejo de arreglos.
- **matplotlib:** para la generación de gráficas de los resultados.
- **openpyxl:** para la exportación de resultados a archivos de Excel.
- **scikit-learn:** utilizada principalmente para apoyo en métricas o validaciones (en caso de uso).

Archivos generados:

El programa genera automáticamente: archivo EVALUACION.xlsx, que contiene los resultados de cada ejecución, matrices de evaluación y la asignación de clusters.

IMPORTANTE: usando *python kmeans.py*, ejecutamos el código en Python desde la terminal.

- ***Orange como Software*** En Orange utilizamos los modulos que se muestran en la figura 20

Figura 20: Modulos utilizados dentro de nuestro flujo de software

La funcion de cada uno se explica a continuación

- **CSV File Import**: Nos ayuda a importar archivos de excel o que no sean .tab
- **Select Columns**: Filtra las columnas por Target, Meta o Included (Tambien existe la columna Ignored si deseamos ignorar por completo la columna)
- **Continuize**: Nos ayuda a la normalizacion de los datos convirtiendo los categoricos a numericos para aplicar la distancia Euclideana
- **K-means**: Aplica el modelo K-means con la distancia euclideana
- **Scatter Plot**: Grafica los resultados de K-Means con las columna que seleccionemos
- **Save data**: Guarda los dats en un archivo (Excel, data, tab, etc)